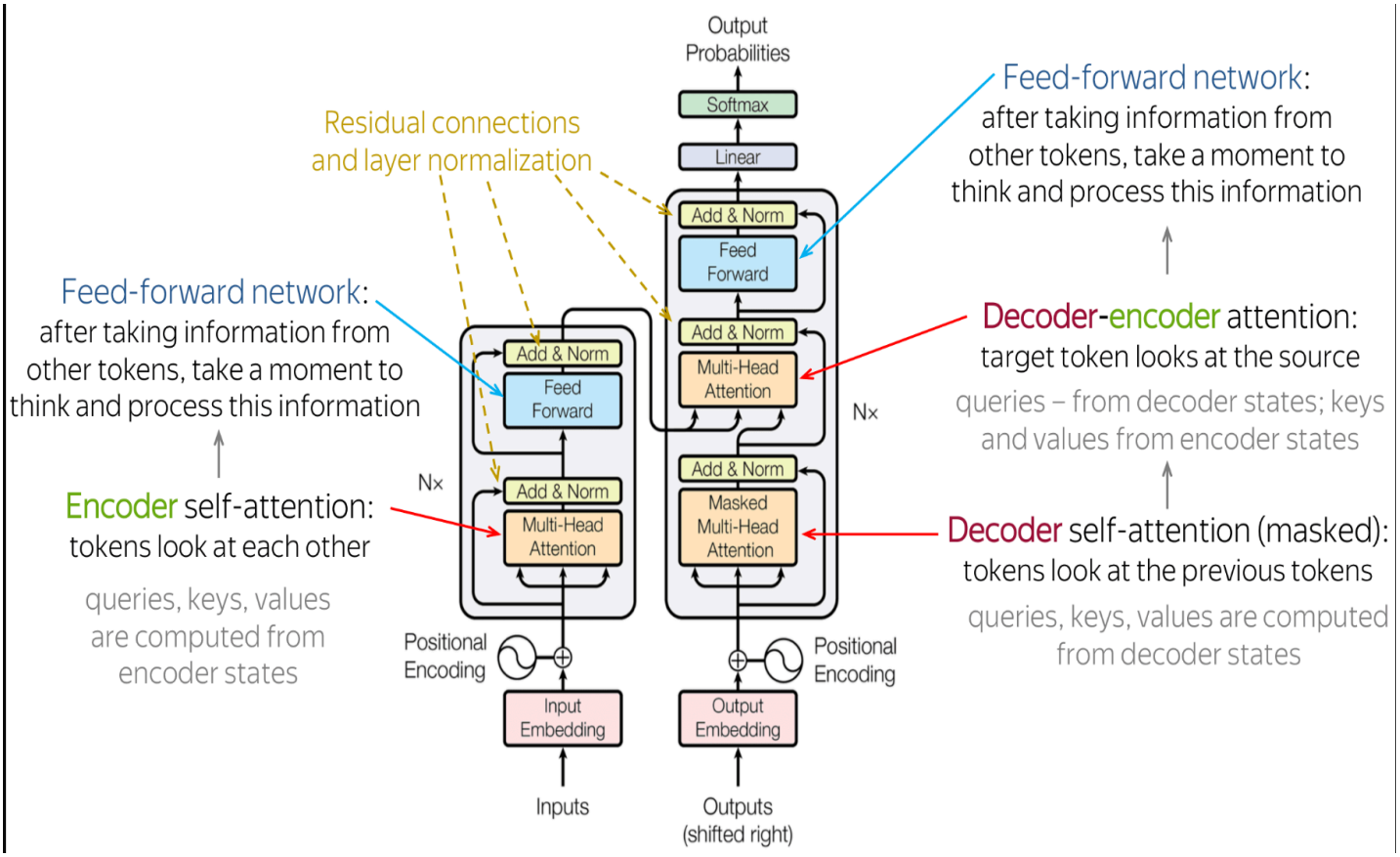
🧠 Transformer Overview

The Transformer is a neural network architecture introduced in the paper *"Attention is All You Need"*. Unlike RNNs and CNNs, Transformers do not process sequences sequentially. Instead, they use a mechanism called **attention** to capture dependencies between tokens in a sequence.

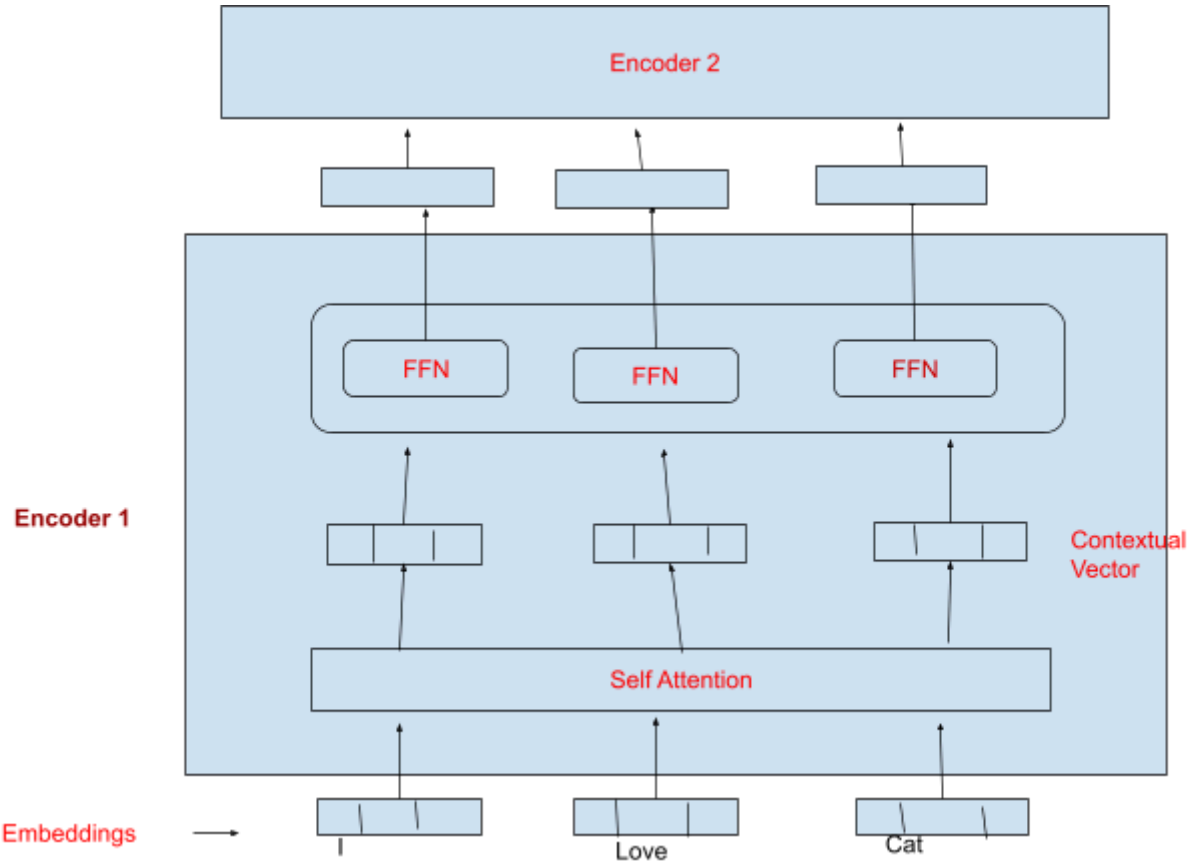
Instead of reading one word at a time like older models (RNNs), the Transformer looks at **all the words at once** and figures out which ones are most important using something called **"attention"**.



🔷 Encoder Structure

Each encoder in the Transformer includes:

1. Input Embedding
2. Positional Encoding
3. Multi-Head Self-Attention
4. Feed-Forward Neural Network (FFN)
5. Residual Connections and Layer Normalization



1. Input Embedding

It converts **words** into **vectors** (i.e., numbers) that the model can understand.

Example:

Sentence: `"I love cats"`

Suppose we use embedding size = 4:

Word	Embedding Vector
I	[0.2, 0.1, 0.5, 0.3]

love	[0.9, 0.6, 0.2, 0.4]
cats	[0.4, 0.7, 0.3, 0.8]

These vectors become the input to the encoder.

2. Positional Encoding

Transformers don't process words in order (like RNNs) , They process tokens in parallel and treat all input positions equally , so we add **positional information** to tell the model the position of each word in the sentence.

Adds a special vector to each word embedding based on its **position in the sentence**.

There are **two main types of positional encoding**:

a)Learned Positional Embeddings

- The position info (like word embeddings) is **learned** during training.
- We use a **lookup table** where each position has its own trainable vector.

Example:
Suppose embedding size = 4 and sequence length = 3
Sentence: `"I love cats"`

Position	Positional Vector (Learned)
0 for i	[0.1, 0.3, 0.2, 0.0]
1 for Love	[0.5, 0.4, 0.7, 0.2]
2 for cats	[0.9, 0.1, 0.3, 0.6]

These vectors are added to the word embeddings:

Word	Word Embedding	+ Positional	Final Vector
I	[0.2, 0.4, 0.3, 0.1]	[0.1, 0.3, 0.2, 0.0]	[0.3, 0.7, 0.5, 0.1]
love	[0.6, 0.2, 0.8, 0.5]	[0.5, 0.4, 0.7, 0.2]	[1.1, 0.6, 1.5, 0.7]
cats	[0.1, 0.6, 0.2, 0.3]	[0.9, 0.1, 0.3, 0.6]	[1.0, 0.7, 0.5, 0.9]

b) **Sinusoidal Positional Encodings**

- These are **fixed** (not learned).
- Uses **sine and cosine** functions of different frequencies.
- Ensures that the model can **extrapolate** to longer sequences.

Formula:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$
$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

Where:
pos = position (e.g., 0, 1, 2…)
i = dimension index (0, 1, 2…)
d_model = embedding size

3. Self Attention :

Self-Attention helps the model understand the **relationship between each word and all the other words** in the sentence — including itself.
It answers the question:
"Which other words should I focus on when understanding this word?"

Why "Self"?
Because:

- The **input and output are from the same sentence**.
- Each word compares itself with **all other words** in the sentence.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- d_k = dimension of keys (scaling factor to avoid large values).

How Self-Attention Works (Step-by-Step):

Example : "The cat sat"

Let's say our input sentence has 3 words, and each word is represented by a 4-dimensional vector:

Word	Vector embeddings

The	**[1, 0, 1, 0]**
cat	**[0, 1, 0, 1]**
sat	**[1, 1, 1, 1]**

✅ **Step 1:** Compute Query (Q), Key (K), and Value (V)

For each word, we compute three vectors:

- **Query (Q):** Represents the word we're focusing on (e.g., "cat").
- **Key (K):** Helps determine how much other words relate to the query.
- **Value (V):** Actual information from other words.

These are obtained by dot product of embeddings with learned weight matrices (W_Q, W_K, W_V).

Lets initialise W_Q, W_K,and W_V with identity metrics

WQ=WK=WV= [1 0 0 0]
| 0 1 0 0 |
| 0 0 1 0 |
[0 0 0 1]

Q(the) = **[1, 0, 1, 0] . W_Q = [1, 0, 1, 0] (because the WQ is identity metrics)**
K(the) = **[1, 0, 1, 0] . W_K = [1, 0, 1, 0]**
V(the) = **[1, 0, 1, 0] . W_V = [1, 0, 1, 0]**

Word	Q	K	V
The	**[1, 0, 1, 0]**	**[1, 0, 1, 0]**	**[1, 0, 1, 0]**
cat	**[0, 1, 0, 1]**	**[0, 1, 0, 1]**	**[0, 1, 0, 1]**
sat	**[1, 1, 1, 1]**	**[1, 1, 1, 1]**	**[1, 1, 1, 1]**

✅ **Step 2 : Calculate Attention Scores**

We measure how much "cat" relates to every other word (including itself) using dot product of Q and K^T:
Example :
Score of
Q(the).K(the) = **[1, 0, 1, 0] . [1, 0, 1, 0] ^{T}=** (1*1+0*0+1*1+0*0) = 2
Q(the).K(cat) = **[1, 0, 1, 0] . [0, 1, 0, 1]^T = 0**
Q(the).K(sat) = **[1, 0, 1, 0] . [1, 1, 1, 1]^T = 2**
This means "the" attends most to itself (score = 2) and "sat" (score = 2).

1. Score for "The":
   2
2. **Score for "cat":**
   [0, 1, 0, 1] · [0, 1, 0, 1]^T = 0+1+0+1 = 2
3. **Score for "sat":**
   [1, 1, 1, 1] · [1, 1, 1, 1]^T = 1*1 + 1*1 + 1*1 + 1*1 = 4

✅ Step 3 : scaling

Scaling in the Attention mechanism is very crucial to prevent the dot product from growing too large .It ensures stable Gradient during the training .
We take up the score and scale down by dividing the sq.root of dk (where d is the dimension)

Here dk = 4 , sqrt(dk) = 2

Score of the = 2/2 = 1
Score of cat = 2/2 = 1
Score of sat = 4/2=2

✅ Step 4 : Apply softmax

Softmax converts raw scores into probabilities (range: 0 to 1, sum = 1). The formula is:

$$\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^{n} e^{x_j}}$$

- Xi= raw score for word i (e.g., 2 for "the").
- n = total number of words (3 in this case).

- e = exponential function (ensures positivity and amplifies differences).

Word	Softmax Weight
The	0.106
cat	0.106
sat	0.788

✅ **Step 5 :** Weighted Sum of Values

Multiply each word's `v` by its softmax weight and sum:

$$\text{Output}_{\text{cat}} = (V_{\text{The}} \times 0.090) + (V_{\text{cat}} \times 0.245) + (V_{\text{sat}} \times 0.665)$$

1. $V_{\text{The}} = [1, 0, 1, 0] \times 0.106 = [0.106, 0, 0.106, 0]$
2. $V_{\text{cat}} = [0, 1, 0, 1] \times 0.106 = [0, 0.106, 0, 0.106]$
3. $V_{\text{sat}} = [1, 1, 1, 1] \times 0.788 = [0.788, 0.788, 0.788, 0.788]$

Total sum = $[0.106,0,0.106,0] + [0,0.106,0,0.106] + [0.788,0.788,0.788,0.788] = [0.894,0.894,0.894,0.894]$

✅ **Step 6: Repeat for All Words**

Do this for every word in the sentence. Now, each word has a representation that considers its context.

Multi-Head Self-Attention

Instead of a single attention mechanism, Transformers use multiple heads:

- Each head captures different aspects of the token relationships.
- Outputs from all heads are concatenated and passed through a linear layer.

Why? One head might capture syntactic dependencies, while another captures semantic ones.

4. Feed-Forward Neural Network (FFN)

- The FFN applies a point-wise nonlinear transformation to each token independently after self-attention.
- Self-attention alone is linear (apart from softmax), so the FFN introduces non-linearity (via activation functions like ReLU/GELU) to help the model learn complex patterns.
- Unlike RNNs, Transformers process all tokens in parallel. The FFN allows each token to be transformed based on its own representation (after attention) without relying on sequential processing.
- FFNs expand dimensions (e.g., 4× the input dimension) and then project back, enabling richer representations.

$$\mathrm{FFN}(x) = \mathrm{ReLU}(xW_1 + b_1)W_2 + b_2$$

5. Residual Connections and Layer Normalization

Add (Residual Connection):

- Purpose: Helps gradients flow directly backward through the network, mitigating vanishing gradients in deep architectures.
- Stabilizes Training: Allows the model to learn small updates (residuals) rather than complete transformations, making deep networks easier to train.
- Preserves Information: Ensures that critical information (e.g., positional encoding or earlier representations) isn't lost after attention/FFN.

Norm (Layer Normalization):

- Purpose: Normalizes the activations across the feature dimension (per token) to stabilize training.
- Faster Convergence: Reduces internal covariate shift by normalizing inputs to layers.
- Robustness to Scale: Makes the model less sensitive to parameter initialization and learning rates.
- Order of Operations: In Transformers, layer norm is typically applied before the attention/FFN (Pre-LN) or after (Post-LN), with Pre-LN being more common in modern architectures for stability.

🔷 Decoder

The decoder has three key components:

A) Masked Multi-Head Self-Attention:

Masked Multi-Head Self-Attention is a crucial component in the **decoder** part of the **Transformer architecture**, especially used in tasks like **language generation** (e.g., translation, text completion).

Goal of Self-Attention in the Decoder

- The model must not "see" future tokens during training or inference.
- To ensure this, we use masked self-attention, which prevents attending to future positions.

Step-by-Step Breakdown

Let's assume we're generating a sequence:
`The cat sat on the` → [MASKED FUTURE]

Let's denote the decoder input tokens as:

[The, cat, sat, on, the]

We want the decoder at position `t` to **attend only to positions ≤ t**.

✅ Step 1: Input Embeddings + Positional Encoding

Each input token is:

- Converted into an embedding vector.
- Positional encoding is added to maintain word order.

✅ Step 2: Linear Projections to Q, K, V

For each attention head, we apply learnable linear projections:

$$Q = E \cdot W_Q, \quad K = E \cdot W_K, \quad V = E \cdot W_V$$

✅ Step 3: Scaled Dot-Product Attention with Masking

This is the **core** attention step.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V$$

Where:

- QK^T: Dot-product between each token's query and all tokens' keys.
- $\frac{1}{\sqrt{d_k}}$: Scaling to avoid large dot products.

The mask M is an upper triangular matrix with values:

- 0 for allowed positions
- -∞ (or a very large negative number) for masked positions

Mask Matrix Example for sequence length 5:
[[0, -∞, -∞, -∞, -∞],
 [0, 0, -∞, -∞, -∞],
 [0, 0, 0, -∞, -∞   ],
 [0, 0, 0, 0,   -∞   ],
 [0, 0, 0, 0,   0   ]]

This ensures each position can only attend to previous positions and itself.

✅ Step 4: Multi-Head Attention

The process is repeated for multiple heads (typically 8-16):

1. Split Q, K, V into h heads (each of dimension d_model/h)
2. Compute scaled dot-product attention for each head independently
3. Concatenate all head outputs
4. Apply final linear projection

✅ Step 5: Add & Norm

Add the input embeddings (residual connection) and apply **Layer Normalization**:

```
Output = LayerNorm(X + MultiHead(Q, K, V))
```

Why Masking is Important

1. Autoregressive Generation: During training, the model should only use previously generated tokens to predict the next token
2. Prevent Data Leakage: Ensures the model doesn't "see" future tokens it's trying to predict
3. Mimics Inference: During inference, the model only has access to previously generated tokens

In Transformers, **Look-Ahead Masking** and **Padding Masking** are two different types of **attention masks** used to control **which tokens are allowed to attend to others** during attention computations. Let's break both down:

1. Look-Ahead Masking (also called Causal Masking)

Purpose: To **prevent the model from seeing future tokens** during **training or inference** in **decoder self-attention**.

During generation, the model should **predict the next word** using only previous words. Look-ahead masking enforces this.

How It Works:
Let's say the sequence is: ["The", "cat", "sat", "on", "mat"]

Then the **look-ahead mask** is a **lower triangular matrix**:

```
[
 [1, 0, 0, 0, 0],  ← "The" can attend to "The"
 [1, 1, 0, 0, 0],  ← "cat" can attend to "The", "cat"
 [1, 1, 1, 0, 0],  ← "sat" can attend to "The", "cat", "sat"
 [1, 1, 1, 1, 0],  ← "on"  can attend to previous tokens
 [1, 1, 1, 1, 1]   ← "mat" can attend to all previous
]
```

Where `1 = allowed`, and `0 = masked` (turned into `-∞` in softmax so attention weight is zero).

✅ Ensures:

- No cheating by looking at future words.
- Enables correct **auto-regressive behavior**.

2. Padding Masking

To **ignore padding tokens** (usually `[PAD]`) in the sequence, which are just fillers to equalize sequence lengths in a batch.

Used in:

- Both **encoder** and **decoder**, in:
  - Self-attention
  - Cross-attention

Padding tokens contain **no useful information**, and the model should **not attend to them**.

B)Cross-Attention

Cross-attention is the mechanism that allows the decoder to focus on relevant parts of the encoder's output while generating each token. It's what connects the encoder and decoder in the Transformer architecture.

Key Characteristics

1. Bridge Between Encoder and Decoder: Lets the decoder access encoder information
2. Query-Key-Value Architecture: Similar to self-attention but with different sources
3. Contextual Generation: Enables output to be conditioned on input

Example: English-to-French Translation:

Input (Encoder): `"The cat sat"`

Output (Decoder so far):
"Le" → decoder is trying to predict next word after "Le"

✅ **1. Encoder: Encode Input :**

Encoder processes: `["The", "cat", "sat"]`

Result: A list of **encoder outputs** (hidden states):
encoder_outputs = [E_1, E_2, E_3] # one vector per input token

✅ **2. Decoder: Masked Self-Attention (for target tokens)**

Decoder so far: `["Le"]`
It runs **masked self-attention** over this to generate a representation for "Le".

✅ **3. Cross-Attention: Connect Decoder to Encoder**
Here:

- The **query (Q)** is from the decoder (e.g., for "Le")
- The **keys (K)** and **values (V)** are from the **encoder outputs**

$$\text{Attention}(Q_{\text{decoder}}, K_{\text{encoder}}, V_{\text{encoder}})$$

This lets the decoder focus on relevant encoder tokens when generating the next token.
For example:

- To generate the next French word `"chat"`, it will attend to the word "cat" from the encoder.

Suppose the decoder wants to generate the second token (`"chat"`). The cross-attention might look like:

Decoder Token: "chat"
Attends to:
"The" → 0.1
"cat" → 0.8 ✅ (strong focus)
"sat" → 0.1

So `"chat"` strongly aligns with `"cat"` in the input.

C)Feed-Forward Network

The Feed-Forward Network (FFN) is a critical component in each decoder layer of the Transformer architecture, providing additional processing power and non-linearity between attention operations.

The FFN in the decoder:

1. Processes each token position independently (position-wise)

2. Applies two linear transformations with a non-linearity in between
3. Helps transform attention outputs into more useful representations for generation